Software Quality 2026: What the Data Shows and What Must Change

Research summary for management | QA Architecture | 22 October 2026

More and worse customer bugs; controls did not keep up

+51%

customer-found bugs

H1 2026 vs H1 2025

+72%

P1/P2 customer-found bugs

H1 2026 vs H1 2025

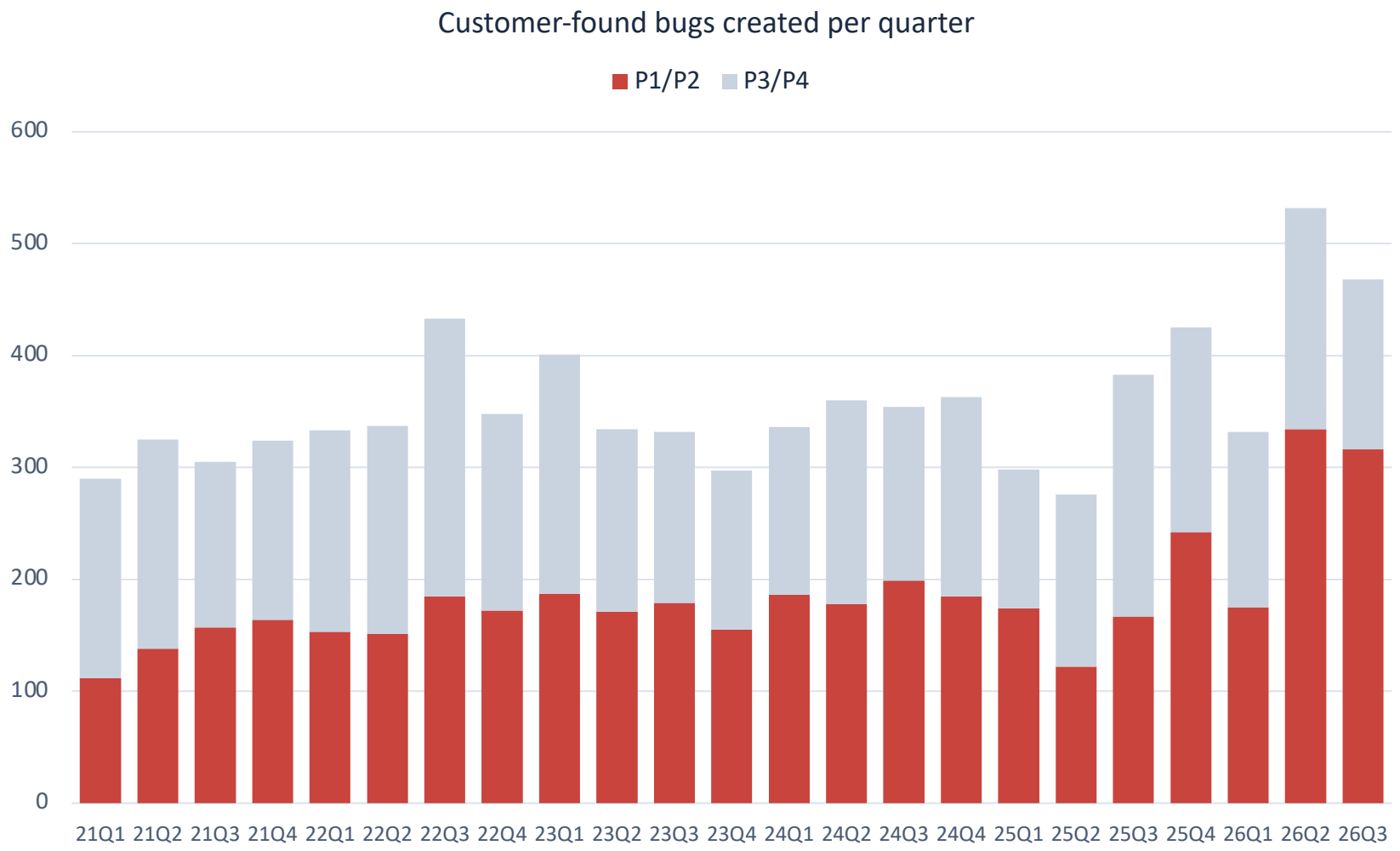
2.4x

more delivered stories

2024-Q3 to 2026-Q3

1. **Not AI:** AI-made changes are not riskier per change (ratio 1.03). AI adds volume.
2. **Real cause:** change volume grew, but quality controls stayed on paper. No check blocks bad changes.
3. **Fix:** add 7 blocking gates, make tests trustworthy, own the hotspot code. Pilot first, review in April 2027.

2026 broke a five-year flat line, and severity rose faster



532

record quarter (2026-Q2)

68%

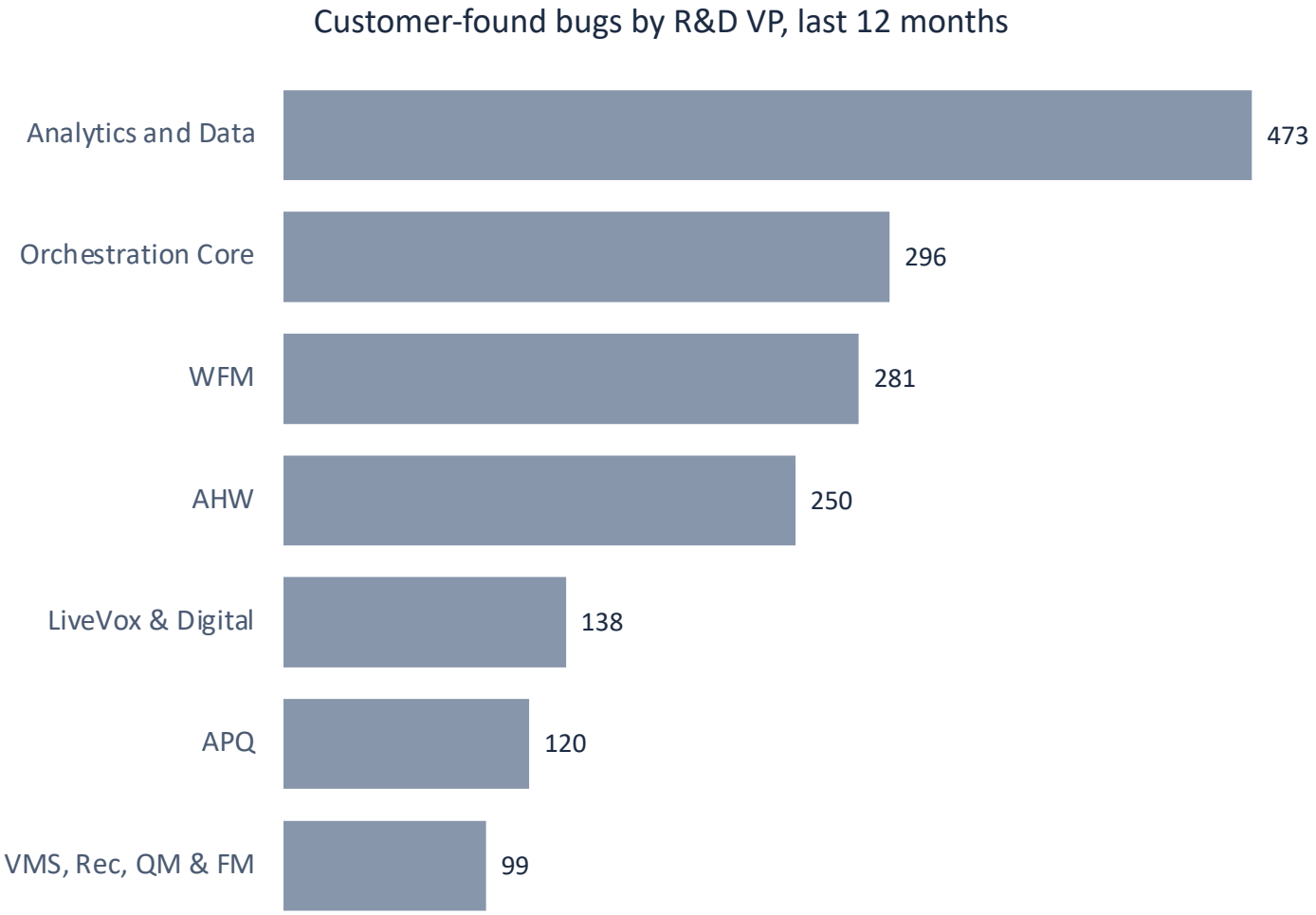
P1/P2 share in 2026-Q3 (about 50% in 2023-24)

~1,300

per year, flat 2021-2025

AI Quality Discovery | Data as of 2026-10-22 | Sources: Jira, Confluence, GitHub

Three R&D domains produce 60% of customer-found bugs



Pilot: Orchestration Core

Contact routing and lifecycle flow

70%

of its bugs are P1/P2

3.8

incidents per bug

+34%

valid bugs in 12 months

Seven causes, ranked by the strength of the evidence

1

Controls exist on paper only

9 of ~15 Jira quality fields < 2% filled; no blocking gate

2

The test signal is broken

Routing E2E passes 1.2% of runs; failures are ignored

3

Real scenarios are not tested

28-31% of escaped bugs

4

State races, monolith vs new services

21% of valid pilot bugs; no per-contact version

5

Big changes into hotspot code

Follow-up fixes 21% (small PRs) to 54% (> 1,000 lines)

6

Knowledge sits with 1-2 people

Bus factor 1 on the VC-to-VC sync code

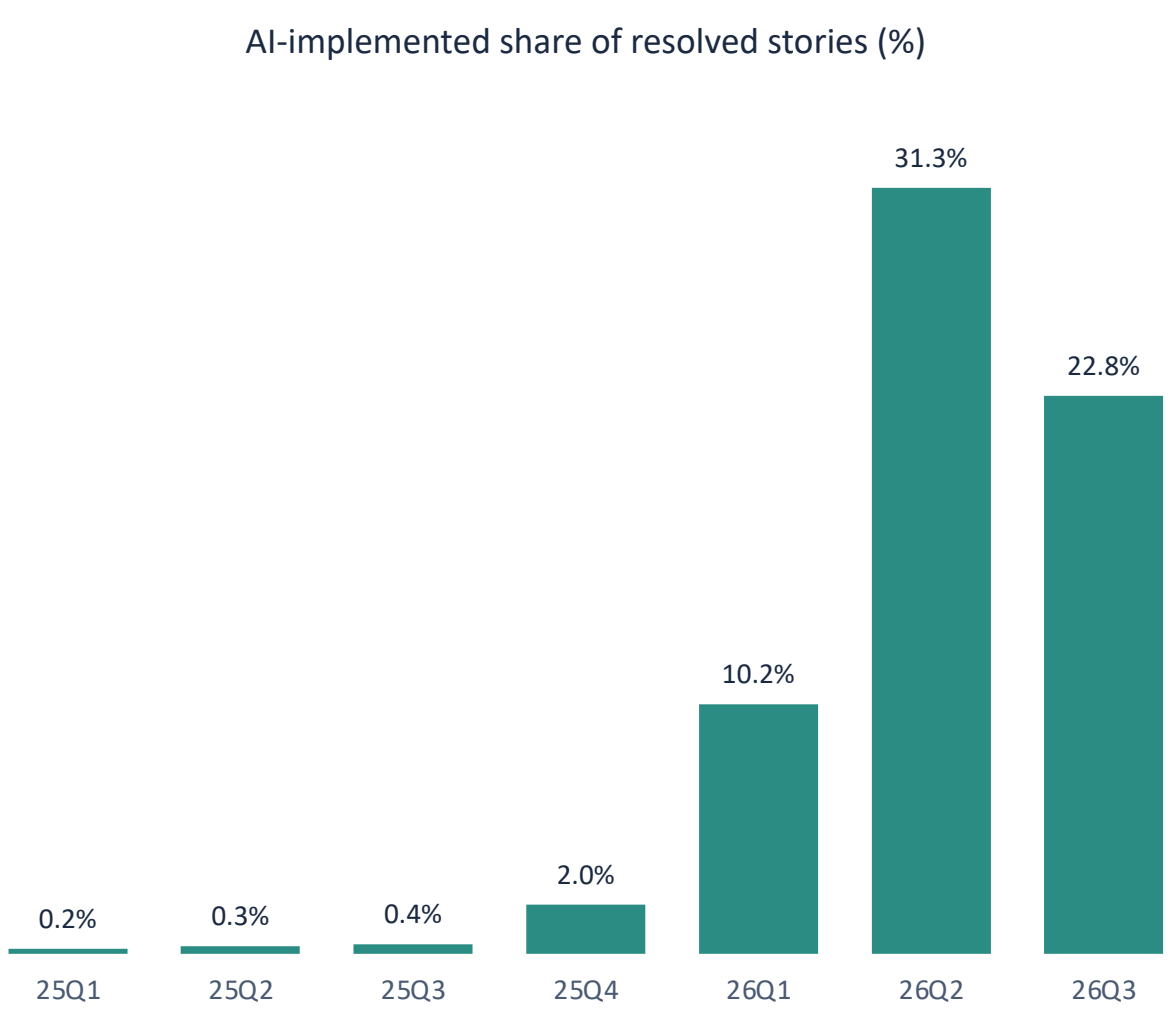
7

Production conditions not reproduced

No performance or chaos test workflow

Not a cause per change: AI-assisted development. AI changes cause bugs at the same rate (ratio 1.03) and need fewer follow-up fixes (0.67).

AI is not riskier per change, but it multiplies change volume



1.03

bug rate ratio, AI vs other changes

95% CI 0.57-1.86

0.67

follow-up fixes after AI PRs vs others

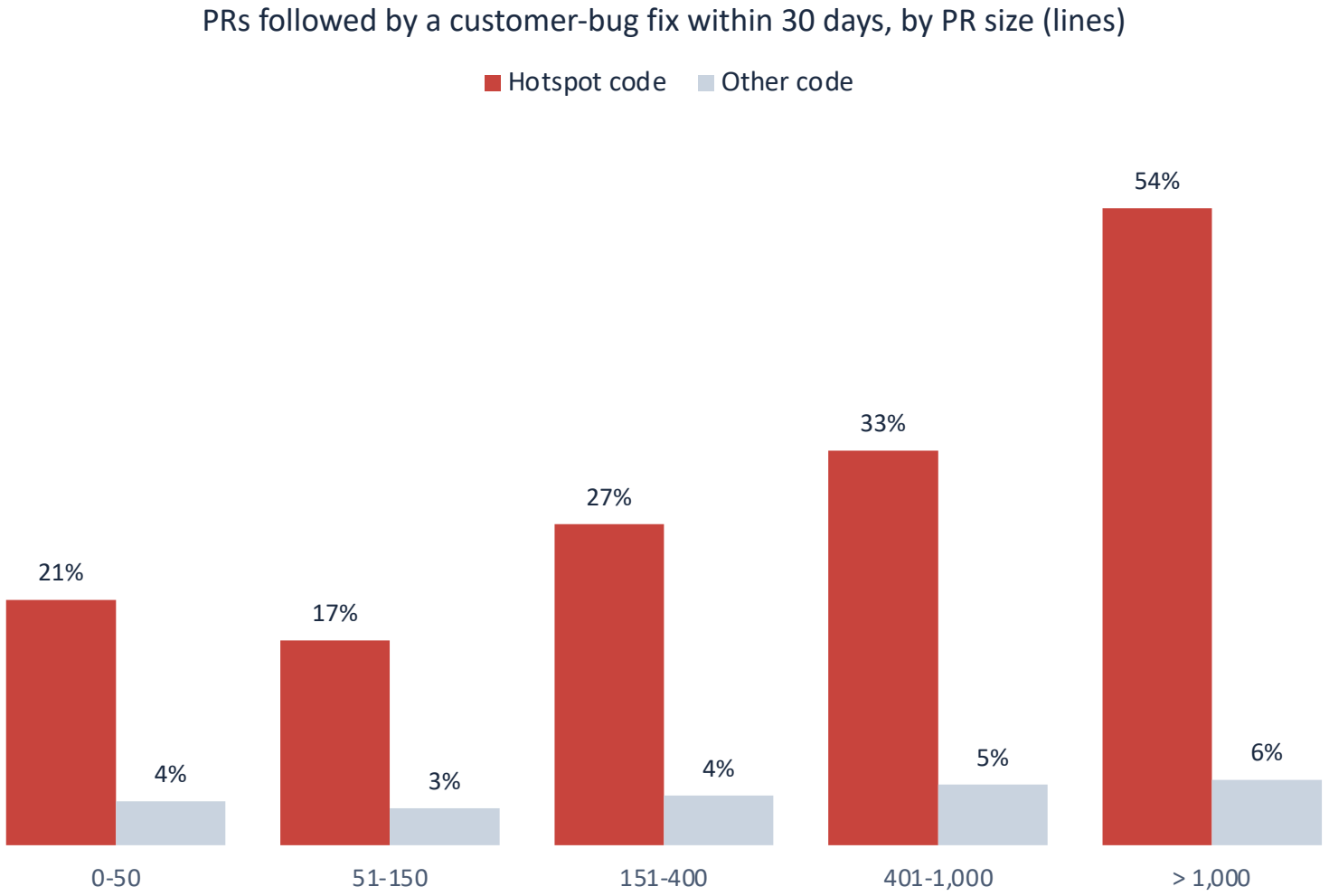
95% CI 0.47-0.95

3.7x

larger AI PRs; review per line not higher

Risk to watch

The real risk: big changes into hotspot code, by anyone



54% vs 6%

large PRs in hotspot code vs other code

39% vs 11%

large non-AI vs large AI PRs: rules must cover all PRs

Review comments per 100 lines fall from 3.3 to 0.03 as PRs grow.

Our controls exist on paper, not in the pipeline

0%

of Stories have a regression impact analysis

9 of 15

Jira quality fields filled on < 2% of items

1.2%

pass rate of the routing E2E pipeline

"TBD"

status on all 26.4 domain Go/No-Go pages

None

code owners file in the monolith

2.7%

of bugs link the change that caused them

Seven gates that block work when a check fails

G1	G2	G3	G4	G5	G6	G7
Design	**Pull request**	**Build**	**Test health**	**Pre-release**	**Deploy**	**Escape**
State, ordering and toggle section; named owners	Hotspot code: <= 400 lines, owner approval, tests, Jira key	Unit, contract and ordering tests; Sonar; secret scan	A suite blocks only at >= 95% pass over 14 days	Fault and load stage; data-based Go/No-Go	Toggle, progressive rollout, monitors green	A customer bug closes only with a linked test

Code and CI (G1-G4)

Release and production (G5-G7)

Not now: regression-impact agents (7% of escapes) and mutation testing (3%). Add later only if the measures show the need.

Also needed: governance gaps the data revealed

1 Governance

Name a sponsor and a quality owner per critical flow; review 7 measures monthly

2 Bug data

51% of pilot bugs invalid; require cause and incident links; dedupe

3 Detection

SLOs and alerts per critical flow; record who detected each bug

4 Migrations

Rollout register; parity checks during dual-run

5 AI governance

One AI label (14 variants today); AI question in root-cause analysis

6 Knowledge

Backup owner per hotspot; design notes for the VC core

7 Security

Secret scanning in Jira and Confluence (credentials found)

8 Data platform

Monthly automated refresh; add Sonar, CI/CD and incident data

End goal: rare escapes, caught early, never repeated

Zero production bugs is not a realistic target. These five measures are, and we already have their baselines.

Fewer escapes

Today

8.9

Target

<= 9

valid pilot bugs per month (13 since May 2026); then -50% vs 2025

No repeats

Today

38%

Target

>= 90%

valid bugs closed with a regression test

Caught first

Today

?

Target

>= 50%

bugs found by monitoring before customers (baseline missing)

Less impact

Today

3.8

Target

<= 2.5

incidents per pilot bug

Trusted tests

Today

1.2-79%

Target

>= 95%

E2E pass rate over 14 days

What we need from management

1. **Name a sponsor and an owner per critical flow**

   No decision owner exists today

2. **Approve the gates for the Orchestration Core pilot**

   Start with test health, PR rules, code owners

3. **Give access to Sonar, CI/CD and incident data**

   Needed to measure detection and to track progress

4. **Require a migration rollout register**

   No production rollout dates exist for 2026 migrations

Timeline

Now	Pilot gates and measures start
2027-Q1	Re-test the AI effect (90-day window)
2027-04	Pilot review against the 5 targets
Then	Extend to Analytics and Data and WFM